E4 Educator v2.0

Tier 3 · Project 3 of 5

P03

Wireless Sensor Network

Walark Electronics · Fundrum Engineering · May 2026

Project overview

The Wireless Sensor Network deploys multiple battery-powered ESP32 sensor nodes that transmit readings to a master E4 via ESP-NOW, with no router or broker required between nodes and master. The master aggregates all node data on a four-card TFT dashboard, forwards readings to MQTT via WiFi, logs everything to SD, and serves a live multi-node web dashboard. Nodes use deep sleep between transmissions for battery efficiency.

New beyond P01/P02:

ESP-NOW star network with up to 4 nodes · NVS MAC address provisioning · Deep sleep node firmware · Node timeout and missed-packet detection · Per-node SD logging · Multi-node MQTT forwarding · Node management web interface · WIFI_AP_STA channel coordination

1 Project specification

1.1 Feature specification

ID	Feature	Detail
F01	ESP-NOW star network	Up to 4 sensor nodes transmit to master via ESP-NOW. No router between nodes and master.
F02	Node deep sleep	Nodes wake, read sensors, transmit, sleep for configurable interval (default 30s). Battery-friendly.
F03	NVS MAC provisioning	Node IDs 1-4 stored in NVS. Master MAC stored in node NVS. Auto-register new nodes on first contact.
F04	AHT20 + LDR per node	Each node carries AHT20 and LDR. Readings packed into SensorPacket.h struct with CRC.
F05	Battery voltage monitor	Node reads own supply voltage via ADC1 divider. Reports battery% in packet. Warns on TFT when low.
F06	Master TFT dashboard	4-card grid: one card per node. Shows temp, humidity, light, battery, last-seen timer, signal quality.
F07	Node timeout detection	Card border turns red after 60s of silence. NO SIGNAL badge after 120s.
F08	Missed packet tracking	Sequence numbers detect missed packets. Miss rate shown on node card and Serial.
F09	Per-node SD logging	Each node's data appended to NODE_N.CSV on master SD. One file per node.
F10	WiFi + MQTT bridge	Master bridges ESP-NOW to MQTT: e4/node/N/temperature etc. WIFI_AP_STA mode for channel co-existence.
F11	Web node dashboard	Multi-node web dashboard: last reading, battery, missed packets per node. Auto-refresh 10s.
F12	Channel management	Master publishes its WiFi channel to MQTT. Nodes can be updated with correct channel via OTA.
F13	OTA — master only	ElegantOTA on master at /update. Nodes updated by flashing directly (USB or separate OTA if WiFi capable).

1.2 Network topology

Device	Role and configuration
Master (E4 Educator)	WIFI_AP_STA mode. ESP-NOW receiver. WiFi client for MQTT. TFT dashboard. SD logger. Web server. OTA.
Node 1–4 (ESP32 DevKit)	WIFI_STA mode. ESP-NOW sender only. AHT20 + LDR. Battery monitor. Deep sleep between transmissions.
MarkPi3	Mosquitto broker receives forwarded node data from master. Node-RED optional dashboard.

2 Node firmware

Each sensor node runs lean firmware: wake from deep sleep, initialise sensors, read values, transmit one ESP-NOW packet, confirm delivery, then sleep again. The entire active period is under 500ms, making battery life measured in weeks on a small LiPo.

2.1 Node packet structure

The SensorPacket.h from T15 is extended with battery voltage and signal quality fields. Both master and node must use the identical struct definition:

```
// SensorPacket.h — shared between all nodes and master
// Must be identical on all devices
#pragma once
#include <Arduino.h>

#define PKT_MAGIC       0xFD
#define PKT_TYPE_SENSOR 0x01
#define PKT_VERSION     0x10

struct __attribute__((packed)) SensorPacket {
  uint8_t  magic;         // 0xFD
  uint8_t  version;       // Protocol version 0x10
  uint8_t  nodeId;        // 1-4
  uint8_t  seqNum;        // Wraps at 255
  float    tempC;         // AHT20 temperature
  float    humidity;      // AHT20 humidity %
  uint16_t lightPct;      // LDR 0-100%
  uint16_t batteryMv;     // Supply voltage in mV
  uint32_t uptimeSec;     // Seconds since node boot/wake
  uint16_t crc;           // Sum of all preceding bytes
};  // 22 bytes total — well within 250-byte ESP-NOW limit

uint16_t calcCRC(const uint8_t* data, size_t len) {
  uint16_t crc = 0;
  for (size_t i = 0; i < len - 2; i++) crc += data[i];
  return crc;
}

bool validateCRC(const SensorPacket& pkt) {
  return pkt.crc == calcCRC((uint8_t*)&pkt, sizeof(pkt));
}
```

2.2 Node platformio.ini

```
[env:e4_node]
platform   = espressif32@6.6.0
board      = esp32dev
framework  = arduino
monitor_speed = 115200

lib_deps =
```

```
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BusIO

build_flags =
  -DFW_VERSION=\"1.0.0\"
  -DFW_DATE=\"2026-05-15\"
  -DNODE_ID=1              ; Change for each node: 1, 2, 3, 4
  -DI2C_SDA=21
  -DI2C_SCL=22
  -DLDR=39                 ; ADC1 — safe with WiFi mode
  -DBATT_ADC=36            ; ADC1 battery voltage divider
  ; Deep sleep interval in microseconds (30 seconds):
  -DSLEEP_US=30000000
  ; Master MAC — UPDATE before flashing each node
  ; Get master MAC by running: WiFi.macAddress() on master
  -DMASTER_MAC_0=0xA4
  -DMASTER_MAC_1=0xCF
  -DMASTER_MAC_2=0x12
  -DMASTER_MAC_3=0x8B
  -DMASTER_MAC_4=0x2E
  -DMASTER_MAC_5=0xF1
  ; WiFi channel — must match master WiFi channel after connection
  -DESPNOW_CHANNEL=6
```

2.3 Node main.cpp — wake, read, send, sleep

```
#include <Arduino.h>
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>
#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include "SensorPacket.h"

uint8_t masterMAC[] = {
  MASTER_MAC_0, MASTER_MAC_1, MASTER_MAC_2,
  MASTER_MAC_3, MASTER_MAC_4, MASTER_MAC_5
};

// Sequence number stored in RTC memory — survives deep sleep
RTC_DATA_ATTR uint8_t seqNum = 0;
RTC_DATA_ATTR unsigned long totalWakes = 0;

Adafruit_AHTX0 aht;
volatile bool sendComplete = false;
volatile bool sendSuccess  = false;

void onSent(const uint8_t* mac, esp_now_send_status_t status) {
  sendSuccess  = (status == ESP_NOW_SEND_SUCCESS);
  sendComplete = true;
}

uint16_t readBatteryMv() {
```

```
  // Voltage divider: 10k/10k on BATT_ADC
  // ADC reading * 3300 / 4095 * 2 (divider ratio)
  int raw = analogRead(BATT_ADC);
  return (uint16_t)((raw * 3300UL / 4095UL) * 2);
}

int readLDRPct() {
  long t = 0;
  for (int i = 0; i < 4; i++) { t += analogRead(LDR); delay(2); }
  return constrain(map(t/4, 200, 3800, 0, 100), 0, 100);
}

void setup() {
  Serial.begin(115200);
  totalWakes++;
  Serial.printf("Node %d wake #%lu FW:%s\n",
                NODE_ID, totalWakes, FW_VERSION);

  // Init sensors
  Wire.begin(I2C_SDA, I2C_SCL);
  bool ahtOk = aht.begin(&Wire);

  // ESP-NOW init
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  esp_wifi_set_channel(ESPNOW_CHANNEL, WIFI_SECOND_CHAN_NONE);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init failed — sleeping.");
    esp_deep_sleep(SLEEP_US); return;
  }
  esp_now_register_send_cb(onSent);

  // Register master as peer
  esp_now_peer_info_t peer = {};
  memcpy(peer.peer_addr, masterMAC, 6);
  peer.channel = ESPNOW_CHANNEL;
  peer.encrypt = false;
  esp_now_add_peer(&peer);

  // Build packet
  SensorPacket pkt = {};
  pkt.magic     = PKT_MAGIC;
  pkt.version   = PKT_VERSION;
  pkt.nodeId    = NODE_ID;
  pkt.seqNum    = ++seqNum;
  pkt.uptimeSec = totalWakes * (SLEEP_US / 1000000UL);
  pkt.batteryMv = readBatteryMv();
  pkt.lightPct  = readLDRPct();

  if (ahtOk) {
    sensors_event_t h, t;
    aht.getEvent(&h, &t);
    pkt.tempC    = t.temperature;
```

```
    pkt.humidity = h.relative_humidity;
  } else {
    pkt.tempC    = -99.0f;
    pkt.humidity = -99.0f;
  }
  pkt.crc = calcCRC((uint8_t*)&pkt, sizeof(pkt));

  // Send and wait for callback (with timeout)
  esp_now_send(masterMAC, (uint8_t*)&pkt, sizeof(pkt));
  unsigned long t0 = millis();
  while (!sendComplete && millis() - t0 < 300) delay(5);

  Serial.printf("Seq:%d T:%.1f H:%.1f L:%d Batt:%dmV %s\n",
    pkt.seqNum, pkt.tempC, pkt.humidity,
    pkt.lightPct, pkt.batteryMv,
    sendSuccess ? "OK" : "FAIL");

  // Sleep
  Serial.flush();
  esp_deep_sleep(SLEEP_US);
}

void loop() {}  // Never reached
```

Key point

RTC_DATA_ATTR variables survive deep sleep. seqNum increments on every wake, giving the master a way to detect missed transmissions. totalWakes multiplied by the sleep interval gives approximate elapsed uptime without an RTC. Always call Serial.flush() before esp_deep_sleep() to ensure the last log line is transmitted.

3 Battery voltage monitoring

Each node monitors its own supply voltage and includes the reading in every packet. This lets the master alert when a node battery is running low, avoiding silent data loss when a node dies.

3.1 Voltage divider circuit

A simple 10k/10k voltage divider scales the battery voltage (up to ~4.2V for LiPo) to the ADC's 3.3V input range:

Node voltage	ADC reading at BATT_ADC (GPIO 36)
4.2V (LiPo full)	~2.1V at divider mid-point → ADC ~2600 → 4200mV reported
3.7V (LiPo nominal)	~1.85V → ADC ~2300 → 3700mV reported
3.0V (LiPo depleted)	~1.5V → ADC ~1860 → 3000mV reported
5V USB power	~2.5V at divider → ADC ~3100 → 5000mV reported (show as — on dashboard)

```
// Battery percentage from voltage (LiPo)
uint8_t batteryPct(uint16_t mv) {
  if (mv >= 4100) return 100;
  if (mv <= 3000) return 0;
  return (uint8_t)map(mv, 3000, 4100, 0, 100);
}

// On master — display battery state
const char* battIcon(uint16_t mv) {
  uint8_t pct = batteryPct(mv);
  if (pct > 70) return "▉▉ OK";
  if (pct > 30) return "▉▒ LOW";
  return "▉▒ !";
}

uint16_t battColour(uint16_t mv) {
  uint8_t pct = batteryPct(mv);
  if (pct > 50) return COL_OK;
  if (pct > 20) return COL_WARN;
  return COL_ERR;
}
```

4 Master firmware

The master E4 runs the most complex firmware in the course. It simultaneously manages: ESP-NOW reception, WiFi connection (WIFI_AP_STA), MQTT forwarding, TFT node-grid display, SD per-node logging, LittleFS web server with multi-node dashboard, and OTA updates.

4.1 WIFI_AP_STA mode and channel locking

The critical configuration that allows ESP-NOW and WiFi to coexist is WIFI_AP_STA mode combined with channel agreement between master and all nodes:

```
// Master network init — must be in this exact order
void initNetwork() {
  // 1. Start in AP_STA mode before anything else
  WiFi.mode(WIFI_AP_STA);

  // 2. Init ESP-NOW BEFORE connecting WiFi
  //    (channels can shift when WiFi connects)
  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init failed."); return;
  }
  esp_now_register_recv_cb(onReceive);
  Serial.printf("Master MAC: %s\n",
                WiFi.macAddress().c_str());

  // 3. Connect WiFi
  WiFi.begin(wifiSSID.c_str(), wifiPass.c_str());
  // ... non-blocking via wifiUpdate() state machine ...
}

// After WiFi connects — publish channel for nodes
void onWiFiConnected() {
  int ch = WiFi.channel();
  Serial.printf("WiFi channel: %d\n", ch);

  // Publish channel so node operators can update node firmware
  char buf[4];
  snprintf(buf, sizeof(buf), "%d", ch);
  mqtt.publish("e4/network/channel", buf, true);

  // Notes for nodes:
  // Nodes must set ESPNOW_CHANNEL to match this value
  // and call esp_wifi_set_channel() before esp_now_send()
}
```

4.2 Node data store and packet processing

```
#define MAX_NODES 4
#define NODE_WARN_MS  60000   // Yellow border after 60s
#define NODE_DEAD_MS 120000   // Red + NO SIGNAL after 120s

struct NodeData {
```

```
  bool     valid      = false;
  float    tempC      = 0;
  float    humidity   = 0;
  uint16_t lightPct   = 0;
  uint16_t batteryMv  = 0;
  uint32_t uptimeSec  = 0;
  uint8_t  seqNum     = 0;
  uint16_t missedPkts = 0;
  unsigned long lastSeen = 0;
  uint8_t  mac[6]     = {};
};

NodeData nodes[MAX_NODES + 1];  // Index 1–4
volatile bool    newData   = false;
volatile uint8_t latestId  = 0;
SensorPacket     latestPkt;

// Receive callback — minimal, runs in WiFi task context
void onReceive(const uint8_t* mac,
               const uint8_t* data, int len) {
  if (len != sizeof(SensorPacket)) return;
  memcpy(&latestPkt, data, sizeof(SensorPacket));
  if (latestPkt.magic != PKT_MAGIC) return;
  if (!validateCRC(latestPkt)) return;
  latestId = latestPkt.nodeId;
  newData  = true;
}

// Process in loop() context
void processPacket() {
  if (!newData) return;
  newData = false;

  uint8_t id = latestId;
  if (id < 1 || id > MAX_NODES) return;

  NodeData& nd = nodes[id];

  // Sequence number check
  if (nd.valid) {
    uint8_t expected = nd.seqNum + 1;
    if (latestPkt.seqNum != expected) {
      uint8_t missed = latestPkt.seqNum - expected;
      nd.missedPkts += missed;
      Serial.printf("Node %d: %d missed (total %d)\n",
                    id, missed, nd.missedPkts);
    }
  }

  nd.valid     = true;
  nd.tempC     = latestPkt.tempC;
  nd.humidity  = latestPkt.humidity;
  nd.lightPct  = latestPkt.lightPct;
  nd.batteryMv = latestPkt.batteryMv;
```

```
  nd.uptimeSec = latestPkt.uptimeSec;
  nd.seqNum    = latestPkt.seqNum;
  nd.lastSeen  = millis();

  Serial.printf("Node%d seq:%d T:%.1f H:%.1f
    L:%d B:%dmV Up:%lus\n",
    id, nd.seqNum, nd.tempC, nd.humidity,
    nd.lightPct, nd.batteryMv, nd.uptimeSec);

  forwardToMQTT(id);
  logNodeReading(id);
  updateNodeGrid();  // Redraw TFT
}

void forwardToMQTT(uint8_t id) {
  if (!mqtt.connected()) return;
  NodeData& nd = nodes[id];
  char topic[32]; char buf[16];

  snprintf(topic, sizeof(topic), "e4/node/%d/temperature", id);
  snprintf(buf, sizeof(buf), "%.1f", nd.tempC);
  mqtt.publish(topic, buf, true);

  snprintf(topic, sizeof(topic), "e4/node/%d/humidity", id);
  snprintf(buf, sizeof(buf), "%.1f", nd.humidity);
  mqtt.publish(topic, buf, true);

  snprintf(topic, sizeof(topic), "e4/node/%d/battery", id);
  snprintf(buf, sizeof(buf), "%d", nd.batteryMv);
  mqtt.publish(topic, buf, true);
}
```

5 TFT four-card node grid

The TFT shows a 2×2 grid of node status cards. Each card is 118×118 pixels, rendered via sprite, with colour-coded border indicating node health.

5.1 Card layout

Row (Y in card)	Content	Detail
Y 4–14	Node ID + age	NODE N in brand blue. Last-seen age in seconds (dim). Border colour = health.
Y 18–40	Temperature	Large (textSize 2). Colour-coded OK/WARN/ERR. ——.— if sensor failed.
Y 44–64	Humidity	Large (textSize 2). Colour-coded OK/WARN/ERR.
Y 68–82	Light bar	Mini horizontal bar + percentage.
Y 86–98	Battery	battIcon() + percentage. Colour = battery health.
Y 102–114	Missed / seq	Miss: N dim text. Grows red if miss rate > 5%.

```
void drawNodeCard(uint8_t id) {
  NodeData& nd = nodes[id];
  unsigned long age = nd.valid ? (millis() - nd.lastSeen) : 999999;

  // Border colour = node health
  uint16_t borderCol = !nd.valid  ? COL_LABEL :
                       age > NODE_DEAD_MS ? COL_ERR :
                       age > NODE_WARN_MS ? COL_WARN : COL_OK;

  spr.fillSprite(COL_BG);
  spr.drawRoundRect(1, 1, 116, 116, 8, borderCol);

  // Header row
  spr.setTextColor(COL_BRAND, COL_BG);
  spr.setTextSize(1); spr.setCursor(6, 5);
  spr.printf("NODE %d", id);
  spr.setTextColor(COL_LABEL, COL_BG);
  spr.setCursor(58, 5);
  if (!nd.valid) { spr.print("waiting"); }
  else if (age > NODE_DEAD_MS) {
    spr.setTextColor(COL_ERR, COL_BG);
    spr.print("NO SIGNAL");
  } else { spr.printf("%lus", age/1000); }

  if (!nd.valid) { spr.pushSprite(cardX(id), cardY(id)); return; }

  // Temperature
  uint16_t tc = (nd.tempC > 28.0f) ? COL_ERR :
                (nd.tempC > 24.0f) ? COL_WARN : COL_OK;
  spr.setTextColor(tc, COL_BG); spr.setTextSize(2);
  spr.setCursor(6, 19);
  if (nd.tempC > -90.0f) spr.printf("%.1fC", nd.tempC);
  else spr.print("--.-C");
```

```
  // Humidity
  uint16_t hc = (nd.humidity > 70.0f) ? COL_ERR :
                (nd.humidity > 60.0f) ? COL_WARN : COL_OK;
  spr.setTextColor(hc, COL_BG); spr.setCursor(6, 44);
  spr.printf("%.1f%%", nd.humidity);

  // Light bar
  spr.setTextColor(COL_LABEL, COL_BG); spr.setTextSize(1);
  spr.setCursor(6, 69);
  spr.printf("Lgt %d%%", nd.lightPct);
  int fw = map(nd.lightPct, 0, 100, 0, 80);
  spr.drawRect(6, 78, 82, 6, COL_LABEL);
  if (fw > 0) spr.fillRect(7, 79, fw, 4, COL_ACCENT);

  // Battery
  spr.setTextColor(battColour(nd.batteryMv), COL_BG);
  spr.setCursor(6, 88);
  spr.printf("%s %d%%", battIcon(nd.batteryMv),
             batteryPct(nd.batteryMv));

  // Missed packets
  uint16_t mCol = (nd.missedPkts > 0) ? COL_WARN : COL_LABEL;
  spr.setTextColor(mCol, COL_BG);
  spr.setCursor(6, 103);
  spr.printf("Miss:%d Seq:%d", nd.missedPkts, nd.seqNum);

  spr.pushSprite(cardX(id), cardY(id));
}

// 2x2 grid positions (Y starts below header bar)
int cardX(uint8_t id) { return (id-1) % 2 == 0 ? 1 : 121; }
int cardY(uint8_t id) { return (id-1) < 2 ? 42 : 162; }

void updateNodeGrid() {
  spr.createSprite(118, 118);
  spr.setColorDepth(16);
  for (uint8_t i = 1; i <= MAX_NODES; i++) drawNodeCard(i);
  spr.deleteSprite();
}
```

6 Per-node SD logging

The master logs each node's data to a separate CSV file on SD: NODE_1.CSV, NODE_2.CSV, NODE_3.CSV, NODE_4.CSV. This keeps data from different physical locations separate and easy to analyse independently.

```
bool sdOk = false;

bool initLogger() {
  sdOk = SD.begin(SD_CS);
  if (!sdOk) { Serial.println("SD mount failed."); return false; }

  // Create header files for each node if not present
  for (int n = 1; n <= MAX_NODES; n++) {
    char fname[16];
    snprintf(fname, sizeof(fname), "/NODE_%d.CSV", n);
    if (!SD.exists(fname)) {
      File f = SD.open(fname, FILE_WRITE);
      if (f) {
        f.println("millis,tempC,humidity,lightPct,battMv,seqNum,missed");
        f.close();
      }
    }
  }
  return true;
}

void logNodeReading(uint8_t id) {
  if (!sdOk || id < 1 || id > MAX_NODES) return;
  NodeData& nd = nodes[id];

  char fname[16];
  snprintf(fname, sizeof(fname), "/NODE_%d.CSV", id);

  char line[80];
  snprintf(line, sizeof(line),
           "%lu,%.2f,%.1f,%d,%d,%d,%d",
           millis(),
           nd.tempC, nd.humidity,
           nd.lightPct, nd.batteryMv,
           nd.seqNum, nd.missedPkts);

  File f = SD.open(fname, FILE_APPEND);
  if (!f) return;
  f.println(line);
  f.close();
}
```

7 Web dashboard — multi-node view

The LittleFS web dashboard shows all four nodes in a responsive card grid. The /api/nodes endpoint returns a JSON object with current readings, battery, age, and missed packet count for all nodes.

```
// /api/nodes endpoint — returns all node data as JSON
server.on("/api/nodes", HTTP_GET, [](AsyncWebServerRequest* req) {
  unsigned long now = millis();
  String json = "{";
  for (int i = 1; i <= MAX_NODES; i++) {
    NodeData& nd = nodes[i];
    if (i > 1) json += ",";
    json += "\"" + String(i) + "\":";
    if (!nd.valid) {
      json += "{\"valid\":false}";
    } else {
      char buf[160];
      snprintf(buf, sizeof(buf),
        "{\"valid\":true,\"temp\":%.1f,\"humid\":%.1f,"
        "\"light\":%d,\"batt\":%d,\"age\":%lu,\"miss\":%d}",
        nd.tempC, nd.humidity, nd.lightPct,
        nd.batteryMv,
        (now - nd.lastSeen) / 1000,
        nd.missedPkts);
      json += buf;
    }
  }
  json += "}";
  req->send(200, "application/json", json);
});
```

8 Deployment guide

8.1 Step-by-step deployment sequence

Step	Action
1	Flash master E4. Run MAC print sketch to get master MAC address. Note it.
2	Flash master with P03 firmware. Master starts, displays 4 empty node cards.
3	After WiFi connects: check Serial for WiFi channel number. Note it.
4	Update MASTER_MAC_x and ESPNOW_CHANNEL in node platformio.ini. Set NODE_ID=1.
5	Flash Node 1. Node wakes, transmits, sleeps. Master card 1 shows first reading.
6	Repeat step 4-5 for nodes 2, 3, 4 (change NODE_ID each time).
7	All four node cards showing live data. Check MQTT Explorer for e4/node/N/ topics.
8	Deploy nodes to physical locations. Check range and verify data continues arriving.
9	Connect battery to each node. Confirm batteryMv reads correctly on TFT card.
10	Monitor for 24h. Check SD log files via web dashboard. Verify no excessive missed packets.

8.2 Range and troubleshooting

Symptom	Cause	Solution
Node card stays empty	Wrong master MAC in node build_flags	Re-run MAC print sketch on master. Update MASTER_MAC_x values.
Works without WiFi, fails when WiFi active	Channel mismatch	Check master Serial for WiFi channel. Update ESPNOW_CHANNEL in node build_flags.
High missed packet rate	Range limit or interference	Move node closer to master. Avoid 2.4GHz congestion. Use directional antenna.
CRC fail messages on master	Packet corruption at range edge	Normal near range limits. Consider reducing sleep interval for retry opportunities.
batteryMv reads 0 or wrong	Missing voltage divider or wrong ADC pin	Confirm 10k/10k divider fitted. Confirm BATT_ADC = 36 (ADC1).

9 Commissioning checklist

☐	Step
☐	Flash master. Run MAC print sketch. Record master MAC address.
☐	Flash master with P03 firmware. Confirm TFT shows 4 empty node cards.
☐	Confirm master WiFi connects. Note channel number in Serial output.
☐	Update node platformio.ini with correct master MAC and WiFi channel.
☐	Flash Node 1 (NODE_ID=1). Confirm master card 1 updates within 30s.
☐	Flash Nodes 2, 3, 4 similarly. Confirm all four cards show live data.
☐	Confirm Serial shows no CRC failures and seqNum incrementing correctly.
☐	Check MQTT Explorer: e4/node/1/temperature through e4/node/4/temperature.
☐	Open web dashboard. Confirm all four node cards visible with correct data.
☐	Connect batteries to nodes. Confirm batteryMv values reasonable (3000-4200mV).
☐	Deploy nodes to intended locations. Monitor for 5 minutes for missed packets.
☐	Check SD: confirm NODE_1.CSV through NODE_4.CSV all contain data rows.
☐	Power-cycle master. Confirm all nodes re-appear within one sleep interval.
☐	Test OTA on master: upload new firmware via /update. Confirm nodes unaffected.

10 Extending the project

- Add node acknowledgement: master sends back a config packet to each node via ESP-NOW confirming receipt and setting the next sleep interval dynamically
- Add DS18B20 probe to some nodes via ONE_WIRE pin for soil or water temperature measurement
- Add BMP280 to master to compare indoor pressure with outdoor nodes for local weather micro-analysis

- Upgrade nodes to solar charging: small 6V panel + TP4056 + LiPo for indefinite outdoor deployment
- Extend to 8 nodes by adding a second master that aggregates 4 more nodes and forwards to the same MQTT broker under e4/node/5-8/ topics
- Add Node-RED alert flow: email or push notification when any node battery falls below 20% or goes silent for more than 5 minutes
- Integrate P02 RTC and daily log rotation: master uses DS3231 to timestamp each node reading with real date/time, per-node log rotates daily